
Concepts of Programming Language

— Functional Programming —

Source:

<https://cs.lmu.edu/~ray/notes/functionalprogramming/>
<https://maryrosecook.com/blog/post/a-practical-introduction-to-functional-programming>

Key Programming Paradigms

- **Imperative programming**: programming with commands (Do this, then do that, etc.)
- **Declarative programming**: programming by saying *what* you want, not *how* it should be computed
- **Structured programming**: programming with nicely nested, crisp, control structures (no "gotos")
- **Object oriented programming**: programming around a society of objects, each with their own behaviors
- **Logic programming**: given a collection of facts and inference rules, an engine answers your questions
- **Functional programming**: programming by composing side-effect-free functions that each do one thing and do it well. Compose these functions together using really cool combining forms.

```
a = 0
def increment():
    global a
    a += 1
```



```
def increment(a):
    return a + 1
```

FP Language Support

Pure functional languages, in which you can only do functional programming. There are no side effects ever, so don't even *think* about imperative programming (Haskell, sort of). In reality, the side-effects are there, but they are “wrapped” into funny containers that are themselves managed in a functional way.



Mostly functional languages, in which imperative features are there if you need them, but the language is intended to be used for functional programming 90% or more of the time (LISP, Scheme, Clojure, Standard ML, OCaml, F#).



Imperative languages with great functional programming support, which may include first-class functions (JavaScript, Scala) or features that are so near first-class functions that they might as well be considered to be so (C#, Ruby, Python).



Imperative languages with minimal functional programming support, in which you may get pointers to functions but that's about it (Ada, C), or let you simulate higher-order functions with a lot of work (Java, pre Java8).

FP Characteristics

- Functions do not have **side-effects** – It doesn't rely on data outside the current function, and it doesn't change data that exists outside the current function. (e.g., no mutable variables, no I/O, no databases). The Clojure language make all values immutable by default.
- Information hiding is handled through **lexical closures** (because there are no shared stateful objects).
- Functions are **first-class** values – they can be created, passed to functions, returned from functions and stored inside data structures.
- Ubiquitous use of **higher-order functions**: functions that can (1) take functions as arguments and (2) return functions.
- **Currying** – decomposing a function that takes multiple arguments into a function that takes the first argument and returns a function that takes the next argument, and so forth for all the arguments.

FP Characteristics

- **Recursion, folding, and powerful collection operators** are used in place of loops
- Each time a function recurses, a new stack frame is created to store the arguments and local values for the current function invocation. If a function recurses a large number of times, it is possible for the interpreter or compiler to run out of memory. **Tail call optimisation** that reuse the same stack frame for their entire sequence of recursive calls.
- **Parallelization** – running the same code concurrently without synchronization.
- **Lazy evaluation** – a compiler technique that avoids running code until the result is needed.
- **Deterministic** – repetitions of a process yield the same result every time.

Why FP Important?

- **Thread-safe**, which is huge!
- Amenable to more **compiler optimizations**, especially in regard to core and thread allocation, and caching of previous results
- Easier to **prove correct**, because the result of calling a specific function with a specific argument is always the same no matter how many times you make that call. This is **referential transparency**.
- Functions as values give programmers more **opportunities for reuse**
- A lot of **Big Data** processing is based on the **MapReduce paradigm** which is straight out of functional programming theory.
- This style is more directly usable on **multicore architectures** (again, no shared state)

filter() – functional vs imperative

```
# Non-Functional (Imperative) Programming Example
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9]
even_numbers = []

for num in numbers:
    if num % 2 == 0:
        even_numbers.append(num)

print(even_numbers) # Output: [2, 4, 6, 8]
```

Filter takes a function and a collection. It returns a collection of every item for which the function returned True

```
# Functional Programming Example
def is_even(num):
    return num % 2 == 0

numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9]
filtered_numbers = list(filter(is_even, numbers))
print(filtered_numbers) # Output: [2, 4, 6, 8]
```

map() – functional vs imperative

```
# Imperative Programming Example
```

```
numbers = [1, 2, 3, 4, 5]
```

```
squared_numbers = []
```

```
for num in numbers:
```

```
    squared_numbers.append(num ** 2)
```

```
print(squared_numbers) # Output: [1, 4, 9, 16, 25]
```

Map takes a function and a collection of items. It makes a new, empty collection, runs the function on each item in the original collection and inserts each return value into the new collection. It returns the new collection.

```
# Functional Programming Example
```

```
def square(x):
```

```
    return x ** 2
```

```
numbers = [1, 2, 3, 4, 5]
```

```
squared_numbers = list(map(square, numbers))
```

```
print(squared_numbers) # Output: [1, 4, 9, 16, 25]
```


Factorial program using loop – imperative

```
def factorial_loop(n):  
    if n < 0:  
        return "Factorial is not defined for negative numbers"  
    result = 1  
    for i in range(1, n + 1):  
        result *= i  
    return result  
  
# Example usage  
n = 5  
result = factorial_loop(n)  
print(f"Factorial of {n} is {result}") # Output: Factorial of 5 is 120
```

Factorial program using recursion – functional

```
function factorial(n) {  
    return n <= 0 ? 1 : n * factorial(n-1)  
}
```

```
factorial(4)  
= 4 * factorial(3)  
= 4 * (3 * factorial(2))  
= 4 * (3 * (2 * factorial(1)))  
= 4 * (3 * (2 * (1 * factorial(0))))  
= 4 * (3 * (2 * (1 * 1)))  
= 4 * (3 * (2 * 1))  
= 4 * (3 * 2)  
= 4 * 6  
= 24
```

Factorial program using tail recursion – functional

```
function factorial(n) {  
  function f(n, a) {  
    return n <= 0 ? a : f(n-1, n*a)  
  }  
  return f(n, 1)  
}
```

```
factorial(4)  
= f(4, 1)  
= f(3, 4)  
= f(2, 12)  
= f(1, 24)  
= 24
```

Factorial program using reduce() – functional

```
def factorial_reduce(n):  
    return reduce(lambda x, y: x * y, range(1, n + 1)) if n > 0 else 1  
  
# Example usage  
n = 5  
result = factorial_reduce(n)  
print(f"Factorial of {n} is {result}") # Output: Factorial of 5 is 120
```

- **Reduce** takes a function and a collection of items. It returns a value that is created by combining the items.

Factorial program using reduce() – functional

```
sentences = ['Mary read a story to Sam and Isla.',  
             'Isla cuddled Sam.',  
             'Sam chortled.']  
  
sam_count = reduce(lambda a, x: a + x.count('Sam'),  
                   sentences,  
                   0)
```

How does this code come up with its initial `a`? The starting point for the number of incidences of `'Sam'` cannot be `'Mary read a story to Sam and Isla.'` The initial accumulator is specified with the third argument to `reduce()`. This allows the use of a value of a different type from the items in the collection.

Closures

```
def outer_function(x):  
    # This is the outer function that takes a parameter x.  
  
    def inner_function(y):  
        # This is the inner function, which is defined inside the outer function.  
        # It captures the variable x from the outer function's scope.  
  
        return x + y  
  
    return inner_function # Return the inner function
```

The **inner_function** "remembers" the value of **x** from its enclosing scope, even after **outer_function** has completed execution.

```
# Example usage of closures  
closure1 = outer_function(10)  
closure2 = outer_function(20)  
  
result1 = closure1(5) # Adds 10 + 5 = 15  
result2 = closure2(5) # Adds 20 + 5 = 25  
  
print(result1) # Output: 15  
print(result2) # Output: 25
```

Currying

```
# Regular, uncurried function that adds two numbers
def add(x, y):
    return x + y

result = add(2, 3) # Result: 5
```

Currying is a useful technique for creating more **flexible** and **reusable** functions.

A **function** that takes **multiple arguments** is transformed into a series of **unary** (single-argument) functions. This allows you to partially apply the function by providing one argument at a time, creating a new function with **reduced arity**.

```
# Curried version of the add function
def curry_add(x):
    def inner(y):
        return x + y
    return inner

# Create a curried version of add
curried_add = curry_add(2)

# Use the curried function to partially apply arguments
result = curried_add(3) # Result: 5
```

Currying using lambda

```
# Curried version of the add function using lambda expressions
curry_add = lambda x: lambda y: x + y

result = curry_add(2)(3) # Result: 5
```

A **function** that takes **multiple arguments** is transformed into a series of **unary** (single-argument) functions. This allows you to partially apply the function by providing one argument at a time, creating a new function with **reduced arity**.

```
# Curried version of the add function
def curry_add(x):
    def inner(y):
        return x + y
    return inner

# Create a curried version of add
curried_add = curry_add(2)

# Use the curried function to partially apply arguments
result = curried_add(3) # Result: 5
```